# GhostPixel: Browser-Based LSB Image Steganography Using JavaScript and WebAssembly

*Bhushan Ladgaonkar*
Dept. of Computer Engineering
Fr. Conceicao Rodrigues College of Engineering
Mumbai, India
10611@crce.edu.in

*Sakshi*
Dept. of Computer Engineering
Fr. Conceicao Rodrigues College of Engineering
Mumbai, India
10895@crce.edu.in

***Abstract*—Steganography conceals the existence of a message rather than its content. This paper presents GhostPixel, a browser-based tool that hides text inside raster images using least significant bit (LSB) substitution. The bit-level encoder and decoder are written in AssemblyScript and compiled to WebAssembly, while a vanilla JavaScript layer handles image I/O through the Canvas API. A 48-bit header (magic number and length) lets the decoder distinguish ordinary images from carriers. The system runs entirely client-side, so neither the cover image nor the secret leaves the user's device. We describe the architecture, data flow, bit layout and memory interface, verify correctness with an independent decoder on a 14-megapixel image, and discuss security limits: LSB embedding is imperceptible but neither encrypted nor robust, and it is detectable by statistical steganalysis.**

***Index Terms*—steganography, least significant bit, WebAssembly, information hiding, cyber security, Canvas API.**

## I. Introduction

Cryptography protects the content of a message, but the presence of an encrypted message can itself attract attention. Steganography addresses this by embedding data inside an innocuous carrier such as an image, so that an observer does not suspect that communication is taking place [1, 2]. It maps to the confidentiality objective of the CIA triad, and it is a dual-use technique: useful for watermarking and privacy, but also abusable for concealing malicious payloads.

Most classroom steganography tools are desktop scripts. We built GhostPixel as a small web application so that it can be demonstrated without installation and without uploading sensitive files. The objectives are: (1) hide and recover UTF-8 text in an image with no visible change; (2) run fully in the browser; (3) keep the core algorithm fast and isolated in WebAssembly; and (4) evaluate its security honestly. The source code is available at `https://github.com/beeth73/ghostpixel`.

## II. Background and Related Work

Image steganography methods are commonly grouped into spatial-domain and transform-domain techniques [2]. LSB substitution, the simplest spatial method, replaces the least significant bit of pixel values with message bits. Because a one-unit change in an 8-bit channel is below human visual sensitivity, the stego image looks identical to the cover. Its weakness is statistical: replacing LSBs equalises the frequencies of value pairs, which the chi-square attack of Westfeld and Pfitzmann exploits [3], and RS analysis estimates the embedded length in colour images [4]. Provos and Honeyman further showed that stego content can be found on a large scale [1].

GhostPixel deliberately uses plain sequential LSB embedding. It is a teaching tool whose goal is to expose the mechanism clearly, together with its limits. Its second aim is to show a practical use of WebAssembly [5], a portable binary format that runs at near-native speed in the browser, for a compute-bound task: bit manipulation over tens of millions of bytes.

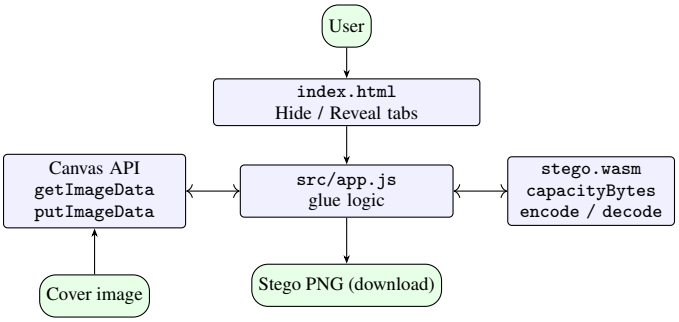



Fig. 1. GhostPixel architecture. All processing happens in the browser; no component communicates with a server.

## III. System Design

### A. Architecture

The application is a static page with no backend. It has three components (Fig. 1). The UI (`index.html`, `style.css`) provides the Hide and Reveal tabs. The glue layer (`src/app.js`) reads a file, draws it to an off-screen canvas, and obtains the RGBA array through `getImageData` [6]. The core (`wasm/index.ts`, compiled to `stego.wasm`) exposes three functions: `capacityBytes`, `encode` and `decode`. After encoding, the modified pixels are written back with `putImageData` and exported as PNG.

### B. Embedding Scheme

Message bit $i$ is stored in the LSB of one colour byte. Alpha bytes are skipped:

$$\text{pixel} = \lfloor i/3 \rfloor, \quad \text{channel} = i \bmod 3, \quad \text{offset} = 4 \cdot \text{pixel} + \text{channel}. \tag{1}$$

Bits are written most-significant-bit first, byte after byte. Fig. 2(a) shows the effect on one pixel: each channel changes by at most one level. Fig. 2(b) shows how consecutive message bits map to the R, G and B bytes of successive pixels. For an

(a) Embedding bits 1, 0, 1 into one pixel

| | R | G | B |
|---|---|---|---|
| Original | 11001010 (202) | 01110001 (113) | 10100110 (166) |
| Message bit | 1 | 0 | 1 |
| Stego | 11001011 (203) | 01110000 (112) | 10100111 (167) |

(b) Mapping of message bits to RGBA bytes

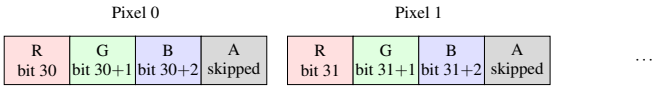



Fig. 2. LSB substitution. (a) The lowest bit (red) of each channel is replaced by a message bit. (b) Three message bits are stored per pixel; the alpha byte is never touched.

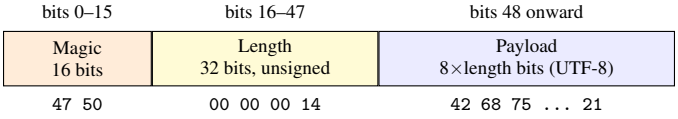



Fig. 3. Bit-stream frame. The hex values shown are those of the test message "Bhushan is the Best!" (20 bytes, `0x14`).

image of width $W$ and height $H$, the payload capacity is

$$C = \left\lfloor \frac{3WH - 48}{8} \right\rfloor \text{ bytes.} \tag{2}$$

For a $4320 \times 3240$ photograph this gives 5,248,794 bytes.

### C. Header and Frame Format

Every message is preceded by a 48-bit header: a 16-bit magic value `0x4750` (ASCII "GP") and a 32-bit unsigned payload length, followed by the UTF-8 payload (Fig. 3). This lets the decoder reject ordinary images and implausible lengths instead of returning garbage text.

### D. Shared Memory Interface

JavaScript and WebAssembly communicate through the module's linear memory (Fig. 4). JavaScript writes the inputs, calls an exported function, then reads the outputs. The RGBA pixel buffer begins at the constant `PIX` = 65,536, defined identically in both `wasm/index.ts` and `src/app.js`; the message bytes follow immediately after the pixels (input for `encode`, output for `decode`). Before copying, JavaScript calls `memory.grow()` if the data would not fit; memory grows in 64 KiB pages. Because JavaScript controls the layout, the module needs no allocator and is built with the stub runtime, which keeps the binary very small.

### E. Hide and Reveal Workflows

Fig. 5 gives the control flow of the two operations. To hide a message, the glue code draws the cover image to an off-screen canvas, reads the RGBA array, forces every alpha byte to 255, encodes the text as UTF-8, copies both into WebAssembly memory and calls `encode`. A return value of 1 means the message fitted and the pixels now carry it; 0 means the message exceeds the capacity. On success the pixels are written back

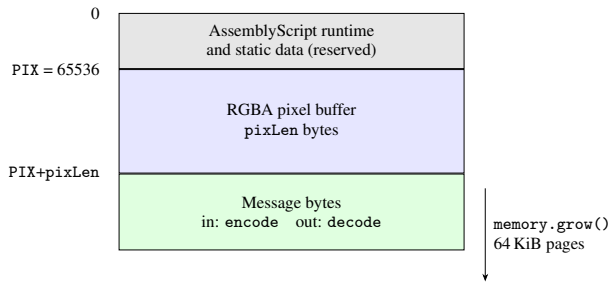



Fig. 4. WebAssembly linear-memory layout shared by `app.js` and `stego.wasm`.

TABLE I
FUNCTIONS EXPORTED BY THE WEBASSEMBLY MODULE

| Function | Parameters | Returns |
|---|---|---|
| `capacityBytes` | `pixLen` | maximum payload size in bytes |
| `encode` | `pixLen, msgLen` | 1 on success; 0 if the message does not fit |
| `decode` | `pixLen` | message length; $-1$ if no message; $-2$ if length is corrupt |

and exported as PNG. To reveal, the PNG is drawn to a canvas, its pixels are copied into WebAssembly memory and `decode` is called; the return value $n$ selects between showing the text, "no hidden message", and "corrupted data".

### F. Decoder Logic

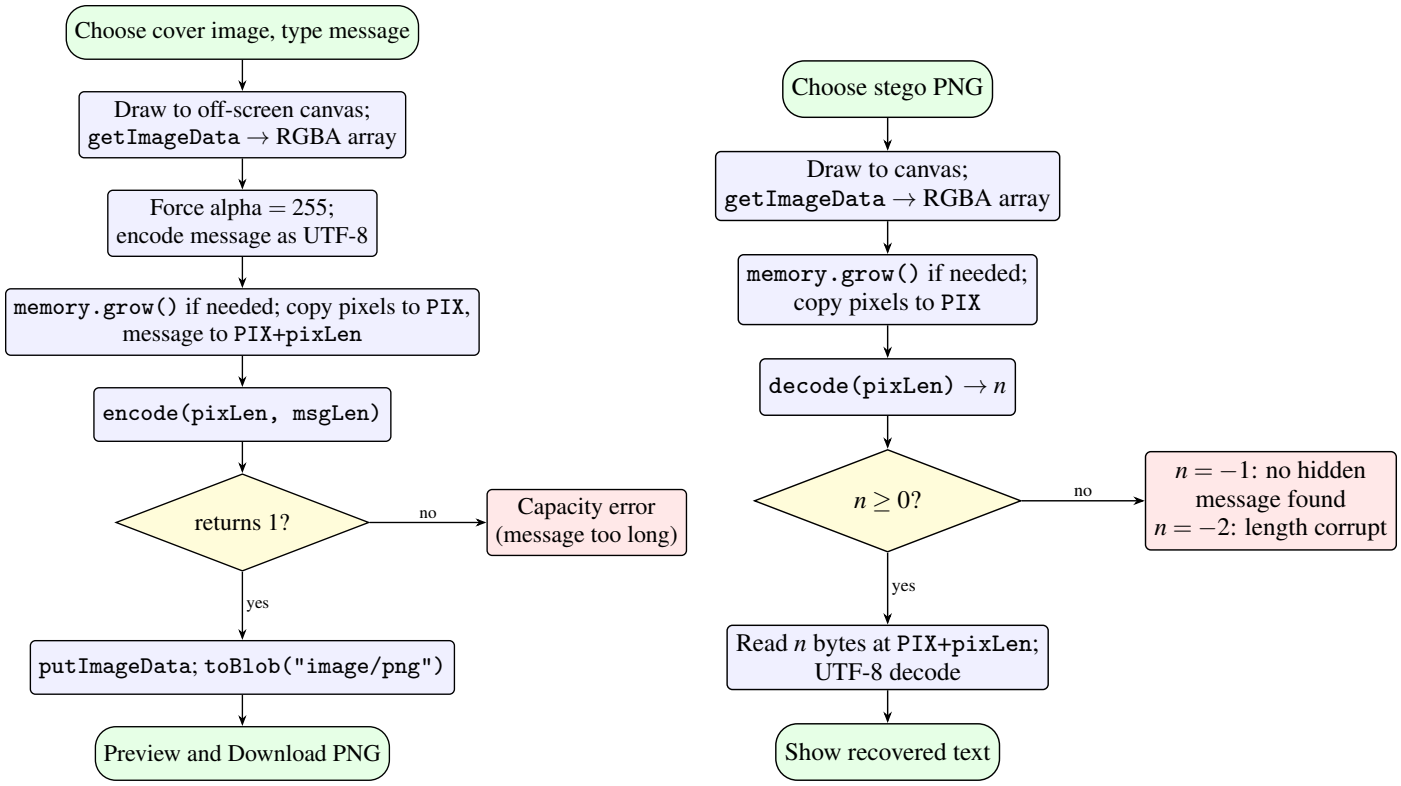
The decoder (Fig. 6) returns $-1$ if the image is too small to hold a header or the magic value is absent, $-2$ if the length field exceeds the image capacity, and otherwise extracts the payload to the output region and returns its length. Table I lists the three exported functions.

### G. Design Decisions

*Alpha handling.* Canvas stores colour premultiplied by alpha, which can silently alter low bits of translucent pixels. The tool therefore forces alpha to 255 before embedding and never writes to alpha bytes. *Lossless output.* Output is always PNG, because JPEG quantisation destroys LSBs. *No allocator.* The memory layout is managed by JavaScript (Section III-D). *AssemblyScript.* It was preferred over Rust for its TypeScript-like syntax, a single npm dependency and a tiny output, with no extra toolchain [7]. *Client-side only.* The secret and the image never leave the device.

## IV. Implementation

The core is about 70 lines of AssemblyScript compiled with `asc -runtime stub -optimize` (`npm run build`). The glue is vanilla JavaScript (ES2020) with no framework or runtime dependency, and the project is organised as `index.html`, `src/app.js`, `src/style.css`, `wasm/index.ts` and the committed `wasm/stego.wasm`. The app is served over HTTP (`npm start`) because browsers block `fetch` of WebAssembly from `file://` URLs. Listings 1 and 2 summarise the encoder and decoder as pseudocode.




(a) Hide (b) Reveal

Fig. 5. Control flow of the two operations as implemented in `src/app.js`. Both share the canvas-to-WebAssembly copy; only `encode` and `decode` run inside the WebAssembly module.

**Listing 1.** Encoder (pseudocode)

```
bitOffset(i) = 4*(i div 3) + (i mod 3)
encode(pixLen, msgLen):
  if msgLen > capacityBytes(pixLen): return 0
  stream = 0x4750 (16 bits) | msgLen (32 bits)
           | message bytes (8 bits each)   // MSB first
  for n, bit in enumerate(stream):
    p[bitOffset(n)] = (p[bitOffset(n)] & 0xFE) | bit
  return 1
```

**Listing 2.** Decoder (pseudocode)

```
decode(pixLen):
  if 48 bits do not fit in the image: return -1
  if read(bits 0..15) != 0x4750:      return -1
  L = read(bits 16..47)
  if L > capacityBytes(pixLen):       return -2
  copy L bytes (bits 48..) to PIX + pixLen
  return L
```

TABLE II
VERIFICATION OF A STEGO IMAGE

| Check | Observed result |
|---|---|
| Hide then reveal in the web app | 20-byte message recovered exactly |
| Independent Python decoder | Header `47 50 00 00 00 14`; same text |
| Image size / alpha | 4320 × 3240 RGBA; alpha 255 everywhere |
| Capacity of that image | 5,248,794 bytes |
| LSB ones ratio, embedded region | 0.356 (74 of 208 bits) |
| LSB ones ratio, untouched region | 0.520 |

The user interface (Fig. 7) shows the capacity of the chosen cover image before the user types, reports the number of bytes hidden or recovered, and offers the result as a PNG download. A footer reminds users to share the output only as PNG.

## V. Results and Security Evaluation

We embedded the 20-byte message "Bhushan is the Best!" in a 4320 × 3240 photograph through the web interface (Fig. 7) and recovered it with the Reveal tab. To confirm that the format is well defined, we decoded the downloaded PNG with a separate Python script that does not share code with the WebAssembly module. Table II summarises the observations and Table III the functional test log.

*Imperceptibility.* Each modified channel differs from the cover by at most one level out of 255, and the stego image is visually indistinguishable from the original (Fig. 7).

*Statistical trace.* In the embedded region the LSB plane *is* the message bit-stream, so its ones ratio (0.356) simply reflects the bit density of the header and ASCII text, in which the top bit of every byte is zero. It is lower than the 0.520 of the untouched region for that reason and is not, by itself, a steganalysis result. Only 208 bits were embedded, so the effect is confined to the first 70 pixels. A payload that fills a large part of the image

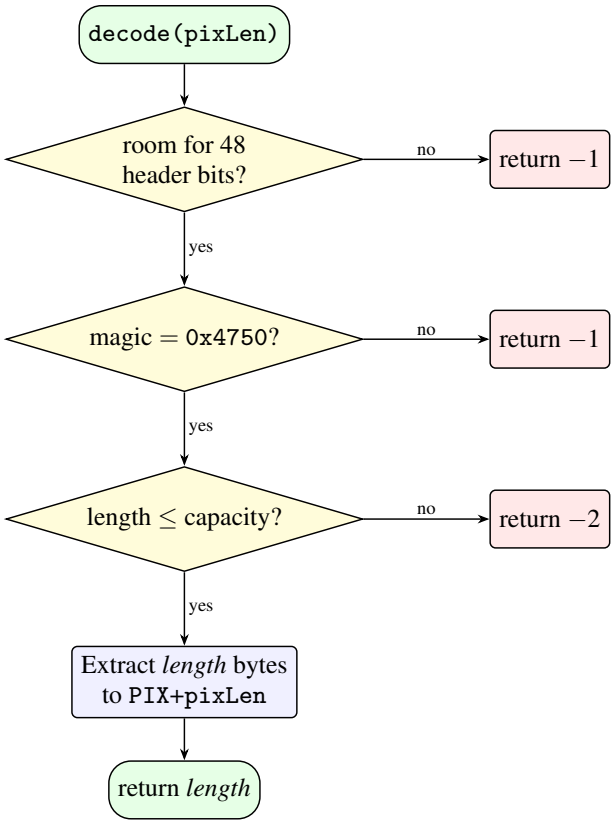



Fig. 6. Decision logic of `decode`.

would flatten the pair-of-values histogram that the chi-square and RS attacks examine [3, 4], and sequential embedding places that signature in one contiguous region. Encrypting the payload would make its bits uniformly random, but would not remove this pair equalisation.

*Robustness.* Lossy re-encoding such as JPEG conversion removes the embedded bits, and the decoder then reports that no message was found (Test 3). This follows from the method and is the stated limitation of the tool.

## VI. Limitations

The payload is hidden but not encrypted, so anyone who knows the format can extract it. Sequential embedding is easy to detect statistically. The method is not robust to compression, resizing or cropping, and only text is supported in the current interface. Forcing alpha to 255 means transparency in a cover image is not preserved, and very large images need enough browser memory to hold a copy of the pixels in WebAssembly memory. The header detects "not a GhostPixel image" and an out-of-range length, but not altered payload bytes. Steganography can conceal harmful data, so the tool is intended for education and should be used lawfully, including under the Information Technology Act, 2000 [8].

## VII. Conclusion and Future Work

GhostPixel demonstrates LSB steganography end to end in the browser, with the performance-critical bit manipulation in WebAssembly and a verified, documented data format. Future work includes encrypting the payload with AES-GCM through the Web Crypto API before embedding (no change to the WebAssembly core is needed, since it handles only bytes), selecting pixels with a key-seeded pseudo-random permutation to resist steganalysis, supporting arbitrary file payloads, adding a CRC32 or HMAC integrity check, a built-in LSB-plane visualiser and chi-square detector, and error-correcting codes so that mild image modification can be survived.

TABLE III
Functional test log

| # | Test | Expected | Result |
|---|---|---|---|
| 1 | Hide, download PNG, reveal | Original text recovered | Pass |
| 2 | Reveal from an ordinary PNG | "No hidden message" | Pass |
| 3 | Convert stego PNG to JPEG, reveal | Message lost | Expected failure |
| 4 | Message larger than capacity | Capacity error | Pass |
| 5 | Independent decode in Python | Header, length, text match | Pass |

TABLE IV
Security properties

| Property | Status | Notes |
|---|---|---|
| Imperceptibility | Yes | At most $\pm 1$ per channel |
| Confidentiality | No | Hidden, not encrypted |
| Steganalysis resistance | Low | Sequential LSB; chi-square, RS |
| Robustness to edits | None | Compression, resize, crop, filters |
| Integrity check | Partial | Magic and length bounds only; payload bytes unprotected |

## References

[1] N. Provos and P. Honeyman, "Hide and seek: An introduction to steganography," *IEEE Security & Privacy*, vol. 1, no. 3, pp. 32–44, 2003.

[2] A. Cheddad, J. Condell, K. Curran, and P. McKevitt, "Digital image steganography: Survey and analysis of current methods," *Signal Processing*, vol. 90, no. 3, pp. 727–752, 2010.

[3] A. Westfeld and A. Pfitzmann, "Attacks on steganographic systems," in *Proc. Information Hiding (IH)*, 1999, pp. 61–76.

[4] J. Fridrich, M. Goljan, and R. Du, "Detecting LSB steganography in color and gray-scale images," *IEEE Multimedia*, vol. 8, no. 4, pp. 22–28, 2001.

[5] A. Haas *et al.*, "Bringing the web up to speed with WebAssembly," in *Proc. ACM PLDI*, 2017, pp. 185–200.

[6] MDN Web Docs, "Canvas API." [Online]. Available: `https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API`

[7] AssemblyScript documentation. [Online]. Available: `https://www.assemblyscript.org/`

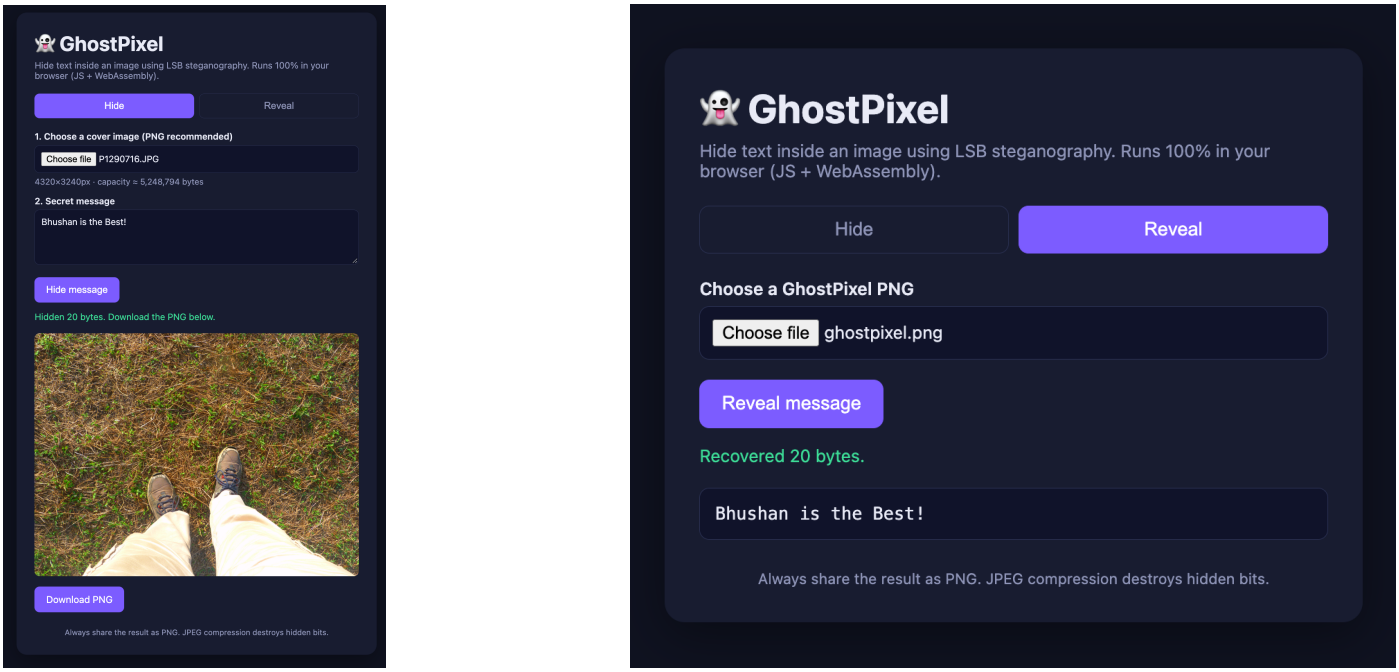



Fig. 7. GhostPixel user interface. Left: Hide tab with a $4320 \times 3240$ cover photograph (capacity 5,248,794 bytes), the secret "Bhushan is the Best!" and the resulting stego preview. Right: Reveal tab recovering the same 20 bytes from the downloaded PNG.

[8] The Information Technology Act, 2000 (India), as amended in 2008.